

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA-0408

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks: 50

CASE STUDY:

1.

Based on the given system, explain in detail the concepts of **logical address and physical address**. Discuss how logical addresses generated by the CPU are translated into physical addresses before accessing memory. Illustrate your answer with a suitable example.

2.

Describe the complete **address translation mechanism using base and limit registers** in the given system. Explain how this method ensures memory protection and prevents processes from accessing unauthorized memory locations.

3.

Explain the concept of **paging** as implemented in the system. Describe how logical memory is divided into pages and mapped to physical frames. Discuss how paging helps in avoiding external fragmentation and improving memory utilization.

4.

Explain the **structure and working of a page table** in detail. Describe how page table entries are used to map logical addresses to physical addresses. Discuss the advantages and limitations of using page tables.

5.

Describe the working of the **Translation Lookaside Buffer (TLB)** in the given system. Explain how TLB reduces memory access time and improves overall system performance.

6.

Differentiate between **TLB hit and TLB miss** in detail. Explain how each case affects memory access time and system efficiency, and illustrate with an example.

7.

Explain how **memory protection** is achieved in the system using base/limit registers and access permissions. Discuss the importance of protection in a multi-user environment and explain what happens when a process violates access rules.

8.

Describe the concept of **segmentation** in detail. Explain how programs are divided into logical segments such as code, data, and stack, and discuss the benefits of segmentation over other memory management techniques.

9.

Explain the **structure and functioning of a segment table**. Describe how logical addresses are translated into physical addresses using segment number and offset.

10.

Compare and contrast **paging and segmentation** in detail. Discuss their differences in terms of memory allocation, fragmentation, flexibility, and performance.

11.

Explain the concept of **virtual memory** and its importance in modern operating systems. Describe how it allows execution of programs larger than the available physical memory.

12.

Describe the **page fault handling process** in detail. Explain each step involved when a page is not found in memory and how the operating system resolves the fault.

13.

Explain the need for **page replacement algorithms** in virtual memory systems. Describe the working of the **FIFO page replacement algorithm** with a suitable example.

14.

Discuss the concepts of **linking and loading** in detail. Explain the differences between static linking and dynamic linking, and describe how loading prepares a program for execution.

15.

Analyze how the combination of **paging, segmentation, TLB, and virtual memory** improves system performance and memory utilization in the given case study. Provide a detailed explanation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
----------	-----------	-------------------------------	--------------------------------	----------------------------------	--------------------------------

Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Logical Address and Physical Address

1. Definition

- A **Logical Address** (Virtual Address) is generated by the CPU during program execution.
- A **Physical Address** is the actual address in the main memory (RAM).
- The **Memory Management Unit (MMU)** converts the logical address into the corresponding physical address before memory access.

2. Need for Address Translation

- Separates user programs from physical memory.
- Protects one process from accessing another process's memory.
- Supports virtual memory.
- Allows efficient memory allocation and relocation.

3. Working Process

1. CPU generates a logical address.
2. The logical address is sent to the MMU.
3. MMU translates it into a physical address.
4. RAM is accessed using the physical address.
5. The requested data is returned to the CPU.

4. Formula and Example

Formula

Example

Given

- Base Register = 5000
- Logical Address = 250

Calculation

Physical Address = 5000 + 250

Answer = 5250

5. Advantages

- Improves memory protection.
- Supports virtual memory.
- Enables process relocation.
- Enhances system security.
- Provides efficient memory utilization.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Devices
- Virtual Memory Systems
- Cloud Computing Platforms

2. Base Register and Limit Register

1. Definition

- The **Base Register** stores the starting physical address allocated to a process.
- The **Limit Register** stores the maximum size (or limit) of the process memory.
- Together, they provide **memory protection** and **address relocation**.

2. Purpose

- Protects one process from accessing another process's memory.
- Checks whether a logical address is within the valid memory range.
- Supports safe memory allocation.
- Prevents unauthorized memory access.

3. Working Process

1. CPU generates a logical address.
2. The logical address is compared with the **Limit Register**.
3. If the logical address is **less than the limit**, it is considered valid.
4. The **Base Register** value is added to the logical address.
5. The resulting value becomes the **Physical Address**.
6. If the logical address exceeds the limit, the operating system generates a **protection trap (error)**.

4. Formula and Example

Formula

Example

Given:

- Base Register = **4000**
- Limit Register = **1000**
- Logical Address = **350**

Calculation:

Physical Address = $4000 + 350$

Physical Address = 4350

Since **350 < 1000**, the address is **valid**.

5. Advantages

- Provides memory protection.
- Prevents illegal memory access.
- Supports process relocation.
- Improves system security.
- Enables efficient memory management.

6. Applications

- Multiprogramming Operating Systems.
- Windows and Linux memory management.
- Embedded systems.
- Real-Time Operating Systems (RTOS).
- Multi-user computing environments.

3. Paging

1. Definition

- **Paging** is a memory management technique that divides the logical memory into fixed-size **pages** and the physical memory into fixed-size **frames**.
- The operating system loads pages into any available frame, allowing a process to occupy **non-contiguous memory locations**.
- Paging helps improve memory utilization and supports virtual memory.

2. Need for Paging

- Eliminates **external fragmentation**.
- Utilizes available memory efficiently.
- Supports **virtual memory** implementation.
- Simplifies memory allocation and management.
- Allows multiple processes to execute simultaneously.

3. Working Process

1. Divide the process into fixed-size **pages**.
2. Divide the main memory into equal-sized **frames**.
3. Store the page-to-frame mapping in the **Page Table**.
4. CPU generates a logical address (Page Number + Offset).
5. The operating system finds the corresponding frame number.
6. The physical address is calculated and memory is accessed.

4. Formula and Example

Formula

Example

Given:

- Page Number = **2**
- Frame Number = **5**
- Page Size = **1024 Bytes**
- Offset = **200**

Calculation:

Physical Address

$$= (5 \times 1024) + 200$$

$$= \mathbf{5120 + 200}$$

$$= \mathbf{5320 \text{ Bytes}}$$

5. Advantages

- Eliminates **external fragmentation**.
- Enables efficient memory utilization.
- Supports virtual memory.
- Allows non-contiguous memory allocation.
- Improves multitasking performance.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Devices
- Cloud Computing
- Virtual Memory Systems
- Multiprogramming Operating Systems

4. Page Table

1. Definition

- A **Page Table** is a data structure maintained by the Operating System.
- It stores the mapping between **logical pages** and **physical frames**.
- Each process has its **own page table** for address translation.
- It is used by the **Memory Management Unit (MMU)** to convert logical addresses into physical addresses.

2. Purpose of Page Table

- Maps logical pages to physical frames.
- Supports virtual memory implementation.
- Enables efficient address translation.
- Allows non-contiguous memory allocation.
- Improves memory utilization.

3. Working Process

1. CPU generates a **logical address**.
2. The logical address is divided into:
 - **Page Number**
 - **Offset**
3. The page number is searched in the **Page Table**.
4. The corresponding **Frame Number** is obtained.
5. The physical address is calculated using the frame number and offset.
6. The required data is accessed from the main memory.

4. Formula and Example

Formula

Page Table

Page Number	Frame Number
0	5
1	8
2	1
3	6

Example

Given:

- Page Number = **2**
- Offset = **30**
- Frame Number = **1**
- Page Size = **100 Bytes**

Calculation:

Physical Address

$$= (1 \times 100) + 30$$

$$= \mathbf{130 \text{ Bytes}}$$

5. Advantages

- Supports virtual memory.
- Eliminates external fragmentation.
- Enables efficient memory allocation.
- Allows processes to occupy non-contiguous memory.
- Improves memory management performance.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Devices

- Virtual Memory Systems
- Cloud Computing
- Multiprogramming Systems

5. Translation Lookaside Buffer (TLB)

1. Definition

- A **Translation Lookaside Buffer (TLB)** is a small, high-speed cache memory inside the **Memory Management Unit (MMU)**.
- It stores recently used **page table entries**.
- The TLB reduces the time required to translate a logical address into a physical address.

2. Purpose of TLB

- Speeds up address translation.
- Reduces page table access time.
- Improves CPU performance.
- Supports efficient virtual memory management.
- Minimizes memory access delay.

3. Working Process

1. CPU generates a **logical address**.
2. The page number is searched in the **TLB**.
3. **If found (TLB Hit):** The frame number is obtained immediately.
4. **If not found (TLB Miss):** The operating system searches the page table.
5. The physical address is calculated.
6. The required data is accessed from the main memory.

4. Formula and Example

Effective Memory Access Time (EMAT)

Where:

- **h** = TLB Hit Ratio
- **T** = TLB Access Time
- **M** = Main Memory Access Time

Example

Given:

- TLB Hit Ratio (h) = **0.90**
- TLB Access Time (T) = **10 ns**
- Memory Access Time (M) = **100 ns**

Calculation:

EMAT

$$= 0.90(10 + 100) + 0.10(10 + 200)$$

$$= 0.90 \times 110 + 0.10 \times 210$$

$$= 99 + 21$$

$$= \mathbf{120\ ns}$$

5. Advantages

- Increases address translation speed.
- Reduces memory access time.
- Improves overall CPU performance.
- Supports efficient virtual memory.
- Reduces page table lookups.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Devices
- Virtual Memory Systems
- Cloud Computing

- Modern Computer Processors

6. TLB Hit and TLB Miss

1. Definition

- **TLB Hit** occurs when the required page number is found in the **Translation Lookaside Buffer (TLB)**.
- **TLB Miss** occurs when the required page number is not found in the TLB, so the operating system must search the **Page Table**.
- TLB Hit provides faster memory access, while TLB Miss increases memory access time.

2. Purpose

- Improve the speed of address translation.
- Reduce memory access time.
- Increase CPU performance.
- Minimize page table lookups.
- Support efficient virtual memory management.

3. Working Process

TLB Hit

1. CPU generates a logical address.
2. The page number is searched in the TLB.
3. The page entry is found.
4. The frame number is obtained immediately.
5. The physical address is generated.
6. Data is accessed from main memory.

TLB Miss

1. CPU generates a logical address.
2. The page number is searched in the TLB.
3. The page entry is not found.

4. The operating system searches the page table.
5. The frame number is obtained and updated in the TLB.
6. The physical address is generated and memory is accessed.

4. Formula and Example

Effective Memory Access Time (EMAT)

Where:

- **h** = TLB Hit Ratio
- **T** = TLB Access Time
- **M** = Main Memory Access Time

Example

Given:

- TLB Hit Ratio = **80% (0.8)**
- TLB Access Time = **10 ns**
- Memory Access Time = **100 ns**

Calculation:

EMAT

$$= 0.8(10 + 100) + 0.2(10 + 200)$$

$$= 0.8 \times 110 + 0.2 \times 210$$

$$= 88 + 42$$

$$= \mathbf{130 \text{ ns}}$$

5. Difference Between TLB Hit and TLB Miss

TLB Hit	TLB Miss
Page found in TLB	Page not found in TLB
Faster address translation	Slower address translation
No page table access required	Requires page table lookup
Lower memory access time	Higher memory access time

6. Advantages and Applications

Advantages

- Improves CPU performance.
- Reduces address translation time.
- Supports efficient virtual memory.
- Decreases overall memory access delay.

Applications

- Windows Operating System
- Linux Operating System
- Android Devices
- Cloud Computing
- Modern Computer Processors

7. Memory Protection

1. Definition

- **Memory Protection** is a mechanism used by the operating system to prevent unauthorized access to memory.
- It ensures that each process can access only its own allocated memory space.
- Memory protection improves system security, stability, and reliability.

2. Purpose of Memory Protection

- Prevent unauthorized memory access.
- Protect one process from interfering with another.
- Improve system security and stability.
- Prevent accidental modification of critical system data.
- Ensure safe execution of multiple processes.

3. Working Process

1. The CPU generates a logical address.
2. The MMU checks the address using the **Base Register** and **Limit Register**.
3. The operating system verifies the access permissions (Read, Write, Execute).
4. If the address is within the valid range, memory access is allowed.
5. If the address exceeds the limit or violates permissions, the operating system generates a **Protection Trap (Exception)**.
6. The invalid memory access is blocked to protect the system.

4. Memory Protection Mechanisms

Base and Limit Register

- Defines the valid memory range for a process.
- Prevents access outside the allocated memory.

Access Protection Bits

- **Read (R)** – Allows reading data.
- **Write (W)** – Allows modifying data.
- **Execute (X)** – Allows executing instructions.

User Mode and Kernel Mode

- **User Mode:** Limited access to system resources.
- **Kernel Mode:** Full access to hardware and memory.

5. Advantages

- Prevents unauthorized memory access.
- Improves system security.
- Protects operating system data.
- Supports safe multitasking.
- Increases system reliability.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Devices
- Multi-user Operating Systems
- Cloud Computing
- Embedded and Real-Time Systems \

8. Segmentation

1. Definition

- **Segmentation** is a memory management technique that divides a program into **variable-sized logical units called segments**.
- Each segment represents a specific part of the program, such as **Code, Data, Stack, or Heap**.
- Every segment has its own **Base Address** and **Limit**, enabling efficient memory organization and protection.

2. Purpose of Segmentation

- Divides a program into logical sections.
- Supports memory protection for each segment.
- Allows independent growth of program segments.
- Improves memory sharing between processes.
- Simplifies program organization and management.

3. Working Process

1. The program is divided into logical segments.
2. Each segment is assigned a **Segment Number**, **Base Address**, and **Limit**.
3. CPU generates a logical address consisting of:
 - Segment Number
 - Offset

4. The operating system searches the **Segment Table**.
5. The offset is checked against the segment limit.
6. If valid, the physical address is calculated and memory is accessed.

4. Formula and Example

Formula

(Applicable only if $\text{Offset} < \text{Limit}$)

Example

Given:

- Segment Number = **2**
- Base Address = **4000**
- Limit = **1000**
- Offset = **250**

Calculation

Physical Address

= Base Address + Offset

= **4000 + 250**

= **4250**

Since **250 < 1000**, the address is **valid**.

5. Advantages

- Supports logical division of programs.
- Provides memory protection for each segment.
- Enables easy sharing of program modules.
- Allows independent growth of segments.
- Improves program organization and flexibility.

6. Applications

- Windows Operating System

- Linux Operating System
- Intel x86 Architecture
- Multitasking Systems
- Compiler and Loader Design
- Virtual Memory Systems

9. Segment Table

1. Definition

- A **Segment Table** is a data structure maintained by the operating system for **segmentation**.
- It stores the **Base Address** and **Limit** of each segment.
- The Memory Management Unit (MMU) uses the Segment Table to translate a logical address into a physical address.

2. Purpose of Segment Table

- Maps logical segments to physical memory.
- Stores the starting address and size of each segment.
- Supports memory protection.
- Enables efficient address translation.
- Allows logical organization of programs.

3. Structure of Segment Table

Segment Number	Base Address	Limit
0	1000	500
1	3000	800
2	5000	600
3	7000	400

- **Base Address:** Starting address of the segment.

- **Limit:** Maximum size of the segment.

4. Working Process

1. CPU generates a **logical address** consisting of **Segment Number** and **Offset**.
2. The operating system searches the **Segment Table** using the Segment Number.
3. The **Base Address** and **Limit** are retrieved.
4. The Offset is compared with the Limit.
5. If **Offset < Limit**, the physical address is calculated.
6. If the Offset exceeds the Limit, the operating system generates a **Protection Trap**.

5. Formula and Example

Formula

(Valid only when Offset < Limit)

Example

Given:

- Segment Number = 2
- Base Address = 5000
- Limit = 600
- Offset = 250

Calculation

Physical Address

$$= 5000 + 250$$

$$= 5250$$

Since **250 < 600**, the address is **valid**.

6. Advantages

- Supports logical memory organization.
- Provides memory protection for each segment.
- Enables efficient address translation.

- Allows independent management of program segments.
- Simplifies program sharing and maintenance.

Applications

- Windows Operating System
- Linux Operating System
- Intel x86 Architecture
- Multitasking Systems
- Compiler and Loader Design
- Virtual Memory Systems

10. Paging vs Segmentation

1. Definition

Paging

- Paging is a memory management technique that divides the logical memory into **fixed-size pages** and the physical memory into **fixed-size frames**.
- It eliminates **external fragmentation** and supports virtual memory.

Segmentation

- Segmentation divides a program into **variable-sized logical segments** such as **Code, Data, Stack, and Heap**.
- Each segment has its own **Base Address** and **Limit**, providing logical organization and memory protection.

2. Purpose

Paging

- Eliminate external fragmentation.
- Improve memory utilization.
- Support virtual memory.
- Enable non-contiguous memory allocation.

Segmentation

- Organize programs into logical units.
- Provide memory protection.
- Allow sharing of program segments.
- Improve program flexibility.

3. Working Process

Paging

1. Divide the program into fixed-size pages.
2. Divide memory into fixed-size frames.
3. Use the **Page Table** to map pages to frames.
4. Calculate the physical address using the frame number and offset.

Segmentation

1. Divide the program into logical segments.
2. Store each segment's **Base Address** and **Limit** in the Segment Table.
3. Generate a logical address (Segment Number + Offset).
4. Calculate the physical address using the base address and offset.

4. Comparison

Paging	Segmentation
Fixed-size pages	Variable-size segments
Uses Page Table	Uses Segment Table
Eliminates external fragmentation	May cause external fragmentation
Simple memory allocation	Logical program organization
Better memory utilization	Better program modularity

5. Advantages

Paging

- Eliminates external fragmentation.
- Efficient memory utilization.
- Supports virtual memory.
- Faster memory allocation.

Segmentation

- Supports logical program structure.
- Provides memory protection.
- Allows segment sharing.
- Easier program maintenance.

6. Applications

Paging

- Windows Operating System
- Linux Operating System
- Android Devices
- Cloud Computing
- Virtual Memory Systems

Segmentation

- Intel x86 Architecture
- Multitasking Operating Systems
- Compiler Design
- Memory Protection Systems
- Embedded Systems

11. Virtual Memory

1. Definition

- **Virtual Memory** is a memory management technique that allows a process to execute even if it is **larger than the available physical memory (RAM)**.

- The operating system uses a portion of the **secondary storage (disk)** as an extension of the main memory.
- It enables efficient execution of large programs by loading only the required pages into RAM.

2. Purpose of Virtual Memory

- Execute programs larger than the available RAM.
- Improve memory utilization.
- Support multitasking by running multiple processes.
- Reduce RAM requirements.
- Increase system efficiency and flexibility.

3. Working Process

1. The CPU requests a page required for execution.
2. The operating system checks whether the page is available in RAM.
3. If the page is present, execution continues.
4. If the page is absent, a **Page Fault** occurs.
5. The operating system loads the required page from secondary storage (disk) into RAM.
6. The program resumes execution after the page is loaded.

4. Example

Given:

- Program Size = **8 GB**
- Available RAM = **4 GB**

Working:

- The operating system loads only the currently required pages into RAM.
- The remaining pages are stored on the disk.
- As execution continues, pages are swapped between RAM and disk whenever needed.

Result:

The **8 GB program** executes successfully using only **4 GB of RAM**.

5. Advantages

- Executes programs larger than physical memory.
- Improves memory utilization.
- Supports multitasking.
- Reduces the need for large physical RAM.
- Increases system performance by managing memory efficiently.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Devices
- Cloud Computing
- Virtual Machines
- Large Database and Scientific Applications

12. Page Fault

1. Definition

- A **Page Fault** occurs when a process tries to access a page that is **not currently available in the main memory (RAM)**.
- The operating system interrupts the process and loads the required page from **secondary storage (disk)** into RAM.
- After loading the page, the process resumes execution.

2. Purpose of Page Fault Handling

- Load the required page into RAM.
- Support virtual memory execution.
- Allow programs larger than RAM to run.

- Improve memory utilization.
- Ensure correct program execution.

3. Working Process

1. CPU generates a logical address.
2. The operating system checks whether the page is present in RAM.
3. If the page is present, execution continues normally.
4. If the page is absent, a **Page Fault** is generated.
5. The required page is loaded from secondary storage into RAM.
6. The page table is updated, and the process resumes execution.

4. Example

Given:

- Process requires **Page 5**.
- RAM currently contains **Pages 1, 2, 3, and 4**.

Working:

- The operating system searches for **Page 5** in RAM.
- Since Page 5 is not available, a **Page Fault** occurs.
- Page 5 is loaded from the disk into RAM.
- The page table is updated.
- Program execution continues.

Result:

The required page is successfully loaded, and the process resumes execution.

5. Advantages

- Supports virtual memory.
- Executes programs larger than RAM.
- Improves memory utilization.
- Loads only the required pages into memory.

- Enables efficient multitasking.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Devices
- Cloud Computing
- Virtual Memory Systems
- Database and Scientific Applications

13. FIFO (First-In, First-Out) Page Replacement Algorithm

1. Definition

- **FIFO (First-In, First-Out)** is a page replacement algorithm used in **Virtual Memory**.
- When the main memory is full, the **page that entered memory first** is removed to make space for the new page.
- It is one of the simplest page replacement algorithms used by the operating system.

2. Purpose of FIFO

- Replace pages when no free frame is available.
- Manage memory efficiently.
- Support virtual memory.
- Reduce memory overflow.
- Improve program execution by loading required pages.

3. Working Process

1. Divide the available memory into fixed-size frames.
2. Load pages into the available frames.

3. When all frames are full, identify the **oldest page**.
4. Remove the oldest page (FIFO rule).
5. Load the new page into the empty frame.
6. Continue the process for each page request.

4. Numerical Example

Given

- Number of Frames = 3
- Page Reference String = 7, 0, 1, 2, 0, 3, 0, 4

FIFO Page Replacement

Page	Frame 1	Frame 2	Frame 3	Status
7	7	-	-	Fault
0	7	0	-	Fault
1	7	0	1	Fault
2	2	0	1	Fault (7 removed)
0	2	0	1	Hit
3	2	3	1	Fault (0 removed)
0	2	3	0	Fault (1 removed)
4	4	3	0	Fault (2 removed)

Result

- Total Page Faults = 7
- Total Page Hits = 1

5. Advantages

- Simple to understand and implement.
- Requires low implementation cost.
- Uses a queue for page management.

- Suitable for basic memory management.
- Easy to maintain.

6. Applications

- Virtual Memory Systems
- Operating Systems
- Embedded Systems
- Educational Simulations
- Basic Memory Management Techniques

14. Linking and Loading

1. Definition

- **Linking** is the process of combining one or more object files and libraries into a single executable program.
- **Loading** is the process of transferring the executable program from secondary storage into the main memory (RAM) for execution.
- Both processes are essential before a program can be executed by the CPU.

2. Purpose of Linking and Loading

- Combine multiple program modules into one executable file.
- Resolve external references between modules.
- Load the executable into main memory.
- Prepare the program for execution.
- Improve modular programming and code reuse.

3. Working Process

Linking

1. Compile the source program into object files.
2. Combine object files and required libraries.

3. Resolve external symbol references.
4. Generate a single executable file.

Loading

1. Load the executable file into RAM.
2. Allocate memory for the program.
3. Perform address relocation if required.
4. Transfer control to the CPU to begin execution.

4. Types of Linking and Loading

Linking

- **Static Linking** – Library code is copied into the executable at compile time.
- **Dynamic Linking** – Library code is linked during program execution.

Loading

- **Absolute Loading** – Program is loaded into a fixed memory location.
- **Relocatable Loading** – Program can be loaded into any available memory location.

5. Advantages

- Supports modular programming.
- Enables code reuse through shared libraries.
- Reduces program development time.
- Improves memory utilization with dynamic linking.
- Simplifies program execution.

6. Applications

- Operating Systems
- Compiler and Linker Design
- Software Development
- Linux and Windows Executables
- Embedded Systems

- Large Modular Applications

15. Integrated Memory Management

1. Definition

- **Integrated Memory Management** is the combination of multiple memory management techniques used by the operating system to improve memory performance and protection.
- It integrates **Paging, Segmentation, Translation Lookaside Buffer (TLB), and Virtual Memory** to manage memory efficiently.
- This approach enables faster address translation, better memory utilization, and secure execution of processes.

2. Purpose of Integrated Memory Management

- Improve overall memory utilization.
- Support virtual memory for large programs.
- Provide memory protection and process isolation.
- Reduce address translation time using TLB.
- Increase overall system performance.

3. Working Process

1. The CPU generates a **logical address**.
2. The **TLB** checks for the required page entry.
3. If the page is found (**TLB Hit**), the physical address is obtained immediately.
4. If not found (**TLB Miss**), the operating system searches the **Page Table** or **Segment Table**.
5. If the page is not available in RAM, **Virtual Memory** loads it from secondary storage.
6. The physical address is generated, and the required data is accessed from the main memory.

4. Components of Integrated Memory Management

- **Paging** – Divides memory into fixed-size pages and frames.
- **Segmentation** – Divides programs into logical segments such as Code, Data, and Stack.
- **TLB** – Stores recently used page table entries for faster address translation.
- **Virtual Memory** – Uses secondary storage to execute programs larger than the available RAM.

5. Advantages

- Improves memory utilization.
- Supports execution of large programs.
- Provides faster address translation.
- Enhances memory protection and security.
- Reduces memory access time.
- Improves overall operating system performance.

6. Applications

- Windows Operating System
- Linux Operating System
- Android Operating System
- Cloud Computing
- Virtual Machines
- Data Centers
- Multi-user Operating Systems